

МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)

**Кафедра математического обеспечения и стандартизации информационных технологий
(МОСИТ)**

КУРСОВАЯ РАБОТА

по дисциплине «Разработка клиент-серверных мобильных приложений»

Тема курсовой работы: «Разработка клиент-серверного мобильного приложения Музыка»

Работу выполнил студент группы ИКБО-65-23 Олефилов Г.Г.
Старший преподаватель Дворникова Е.М.

Цель проекта

Разработка клиент-серверного Android-приложения «MuseFlow» на языке Kotlin, которое даёт пользователю возможность прослушивать музыкальные треки, искать треки по названию или исполнителю, создавать, редактировать и удалять собственные плейлисты, добавлять и удалять треки из плейлистов, использовать приложение в офлайн-режиме благодаря кэшированию, переключать светлую и тёмную тему.

Задачи проекта

- Описание предметной области мобильного приложения.
- Сравнительный анализ существующих аналогов мобильного приложения.
- Обоснование выбора инструментальных средств разработки мобильного приложения.
- Определение функциональных и нефункциональных требований.
- Проектирование пользовательского интерфейса мобильного приложения.
- Логическая модель мобильного приложения (диаграмма в нотации IDEF0, диаграмма классов).
- Проектирование базы данных мобильного приложения.
- Разработка клиентской части мобильного приложения.
- Разработка серверной части мобильного приложения.
- Тестирование разработанного мобильного приложения.

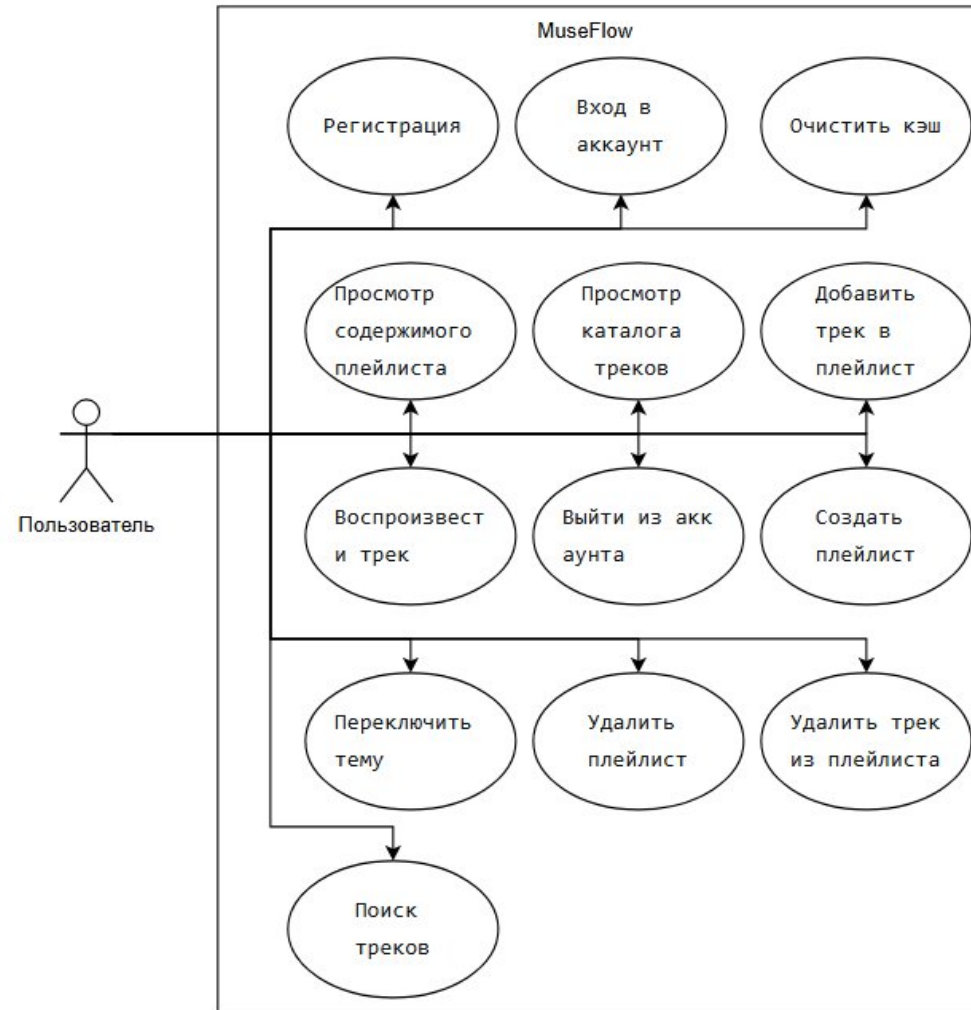
Состояние исследуемой области

В эпоху цифровых технологий пользователи всё чаще сталкиваются с необходимостью удобного доступа к музыкальному контенту — от поиска новых треков до создания личных плейлистов. Существующие платформы, такие как Яндекс.Музыка, VK Музыка, Spotify и Apple Music, предлагают различные подходы к решению этой задачи, сочетая каталоги треков, персональные рекомендации и социальные функции. Однако многим пользователям не требуется громоздкий функционал: вместо сложных алгоритмов им достаточно простого и удобного инструмента для поиска музыки, создания плейлистов и прослушивания треков в офлайн-режиме.

Сравнительный анализ

Компонент	Технология	Назначение
Язык клиента	Kotlin	Основной язык разработки Android
UI-фреймворк	Jetpack Compose + Material 3	Декларативное построение интерфейса
Внедрение зависимостей	Dagger Hilt	Управление зависимостями, тестирование
Локальное хранение	Room	Кэширование треков (офлайн-доступ)
Сетевые запросы	Retrofit + OkHttp + Gson	HTTP-взаимодействие с сервером
Медиаплеер	ExoPlayer	Воспроизведение аудио
Фоновое воспроизведение	MediaSessionService (Media3)	Управление плеером в фоне
Загрузка изображений	Coil	Отображение обложек треков
Асинхронность	Coroutines + StateFlow	Потокобезопасное управление состоянием UI
Язык сервера	Kotlin (Ktor)	Единый стек с клиентом
ORM для сервера	Exposed	Типобезопасные SQL-запросы
База данных	PostgreSQL	Хранение пользователей, треков, плейлистов
Аутентификация	JWT (auth0) + BCrypt	Stateless-авторизация и хеширование паролей

Диаграмма use-case



Архитектура предлагаемого решения

- **Основные компоненты системы:**
- **Клиентская часть (Kotlin + Jetpack Compose)** — отвечает за пользовательский интерфейс, обработку ввода, навигацию между экранами, локальное кэширование (Room) и взаимодействие с сервером через API (Retrofit).
- **Серверная часть (Ktor)** — обрабатывает входящие запросы от клиента, реализует бизнес-логику, обеспечивает JWT-аутентификацию и осуществляет доступ к данным через ORM Exposed.
- **База данных (PostgreSQL)** — хранит сведения о пользователях, музыкальных треках, плейлистах и связях между ними. Обеспечивает целостность и постоянную доступность данных для серверной части.

Основные компоненты

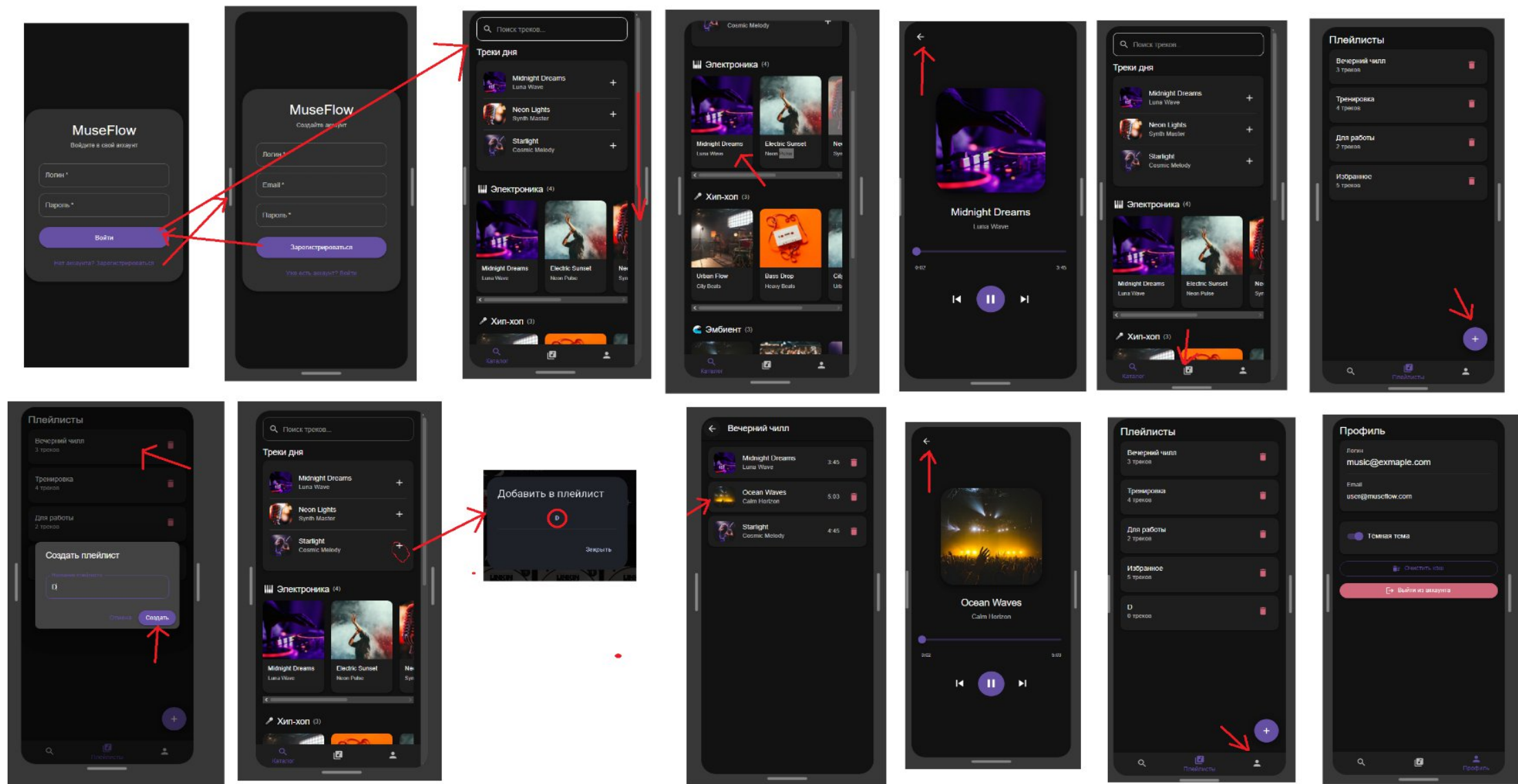


Диаграмма IDEF0 (контекстный уровень)

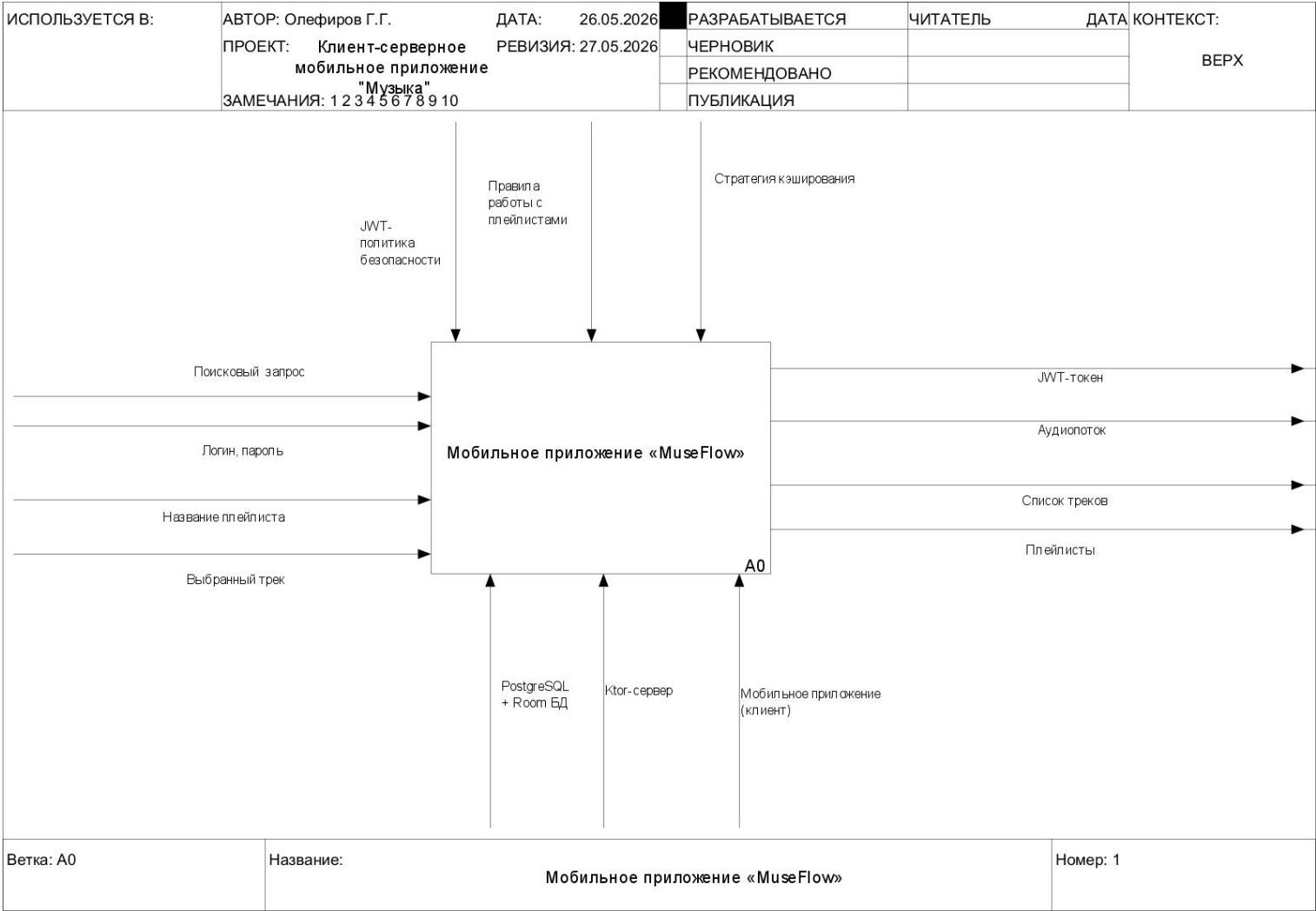
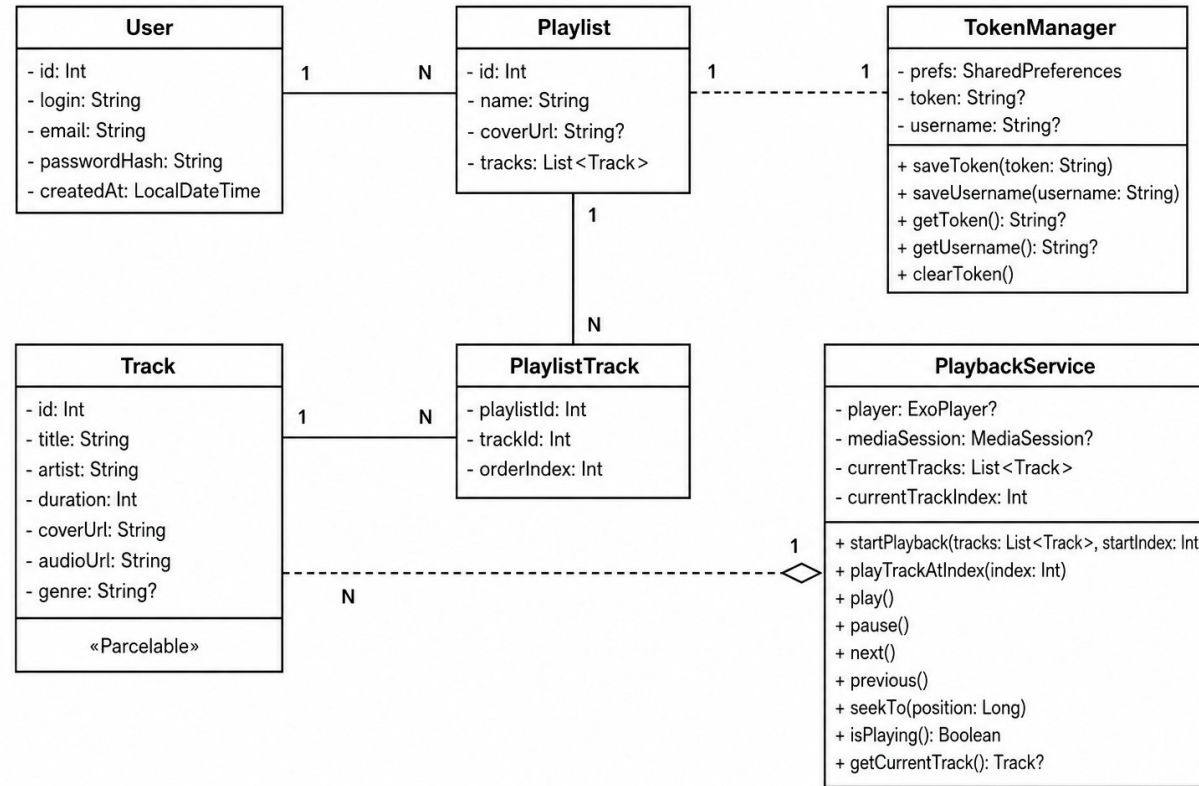
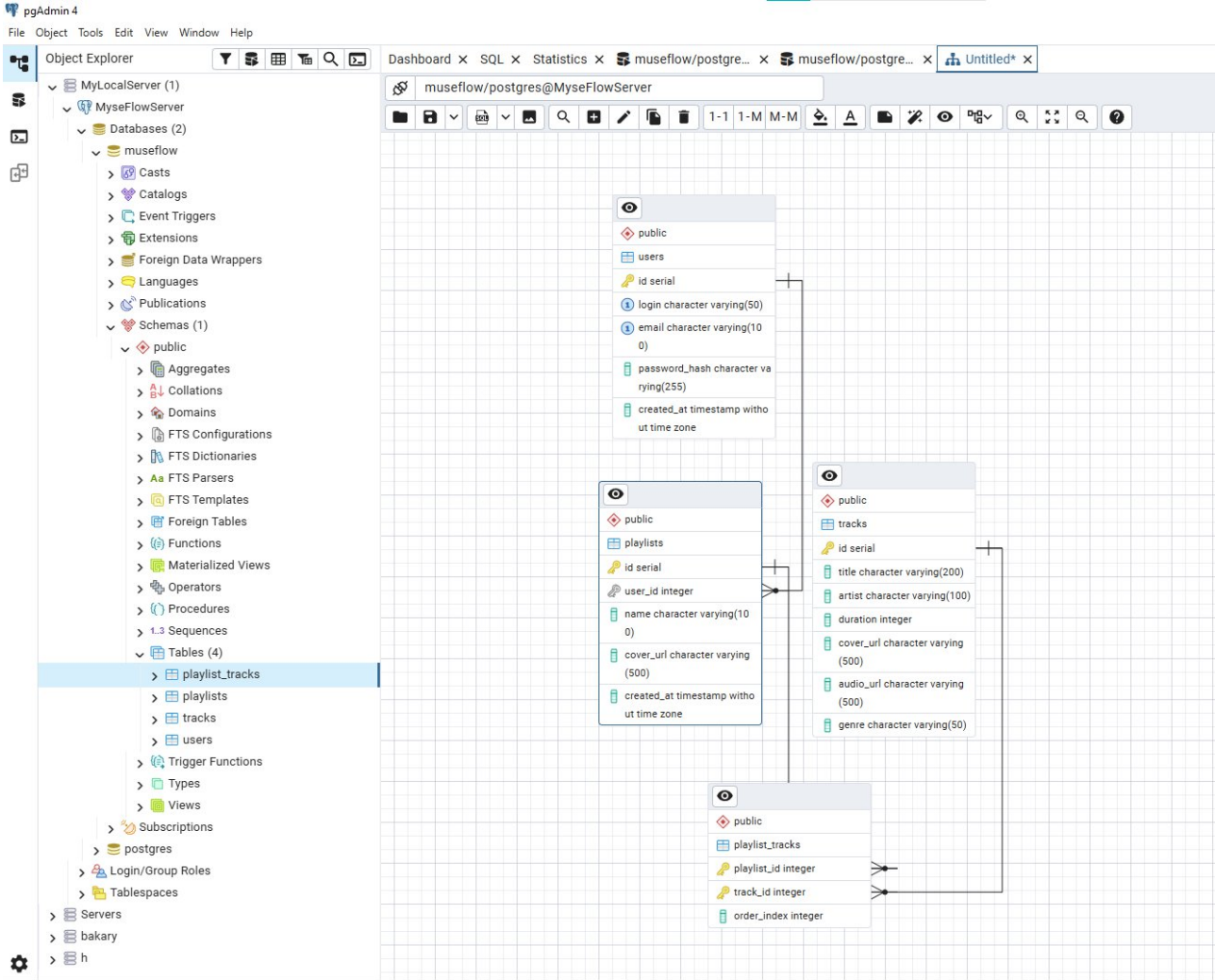


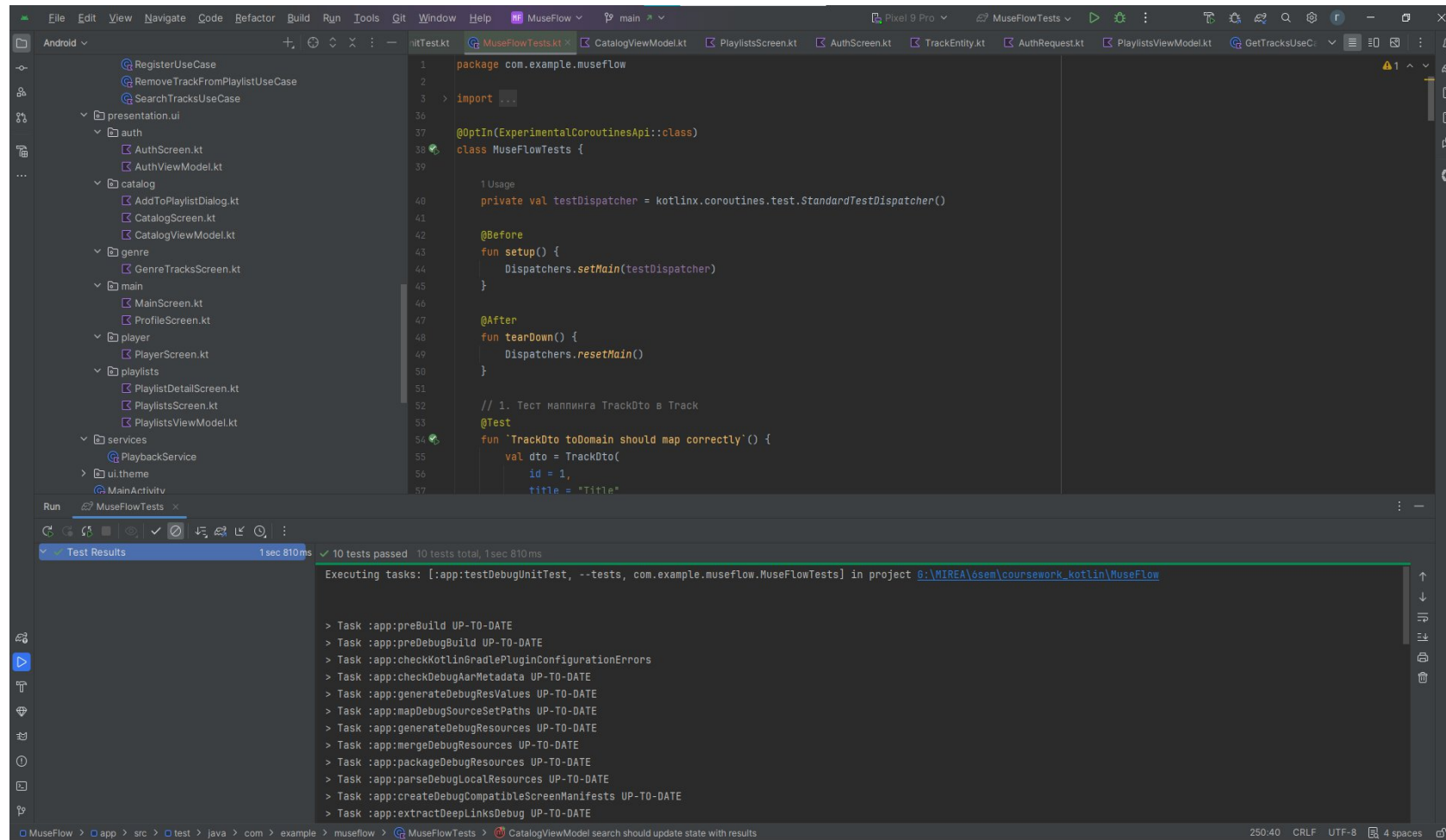
Диаграмма классов



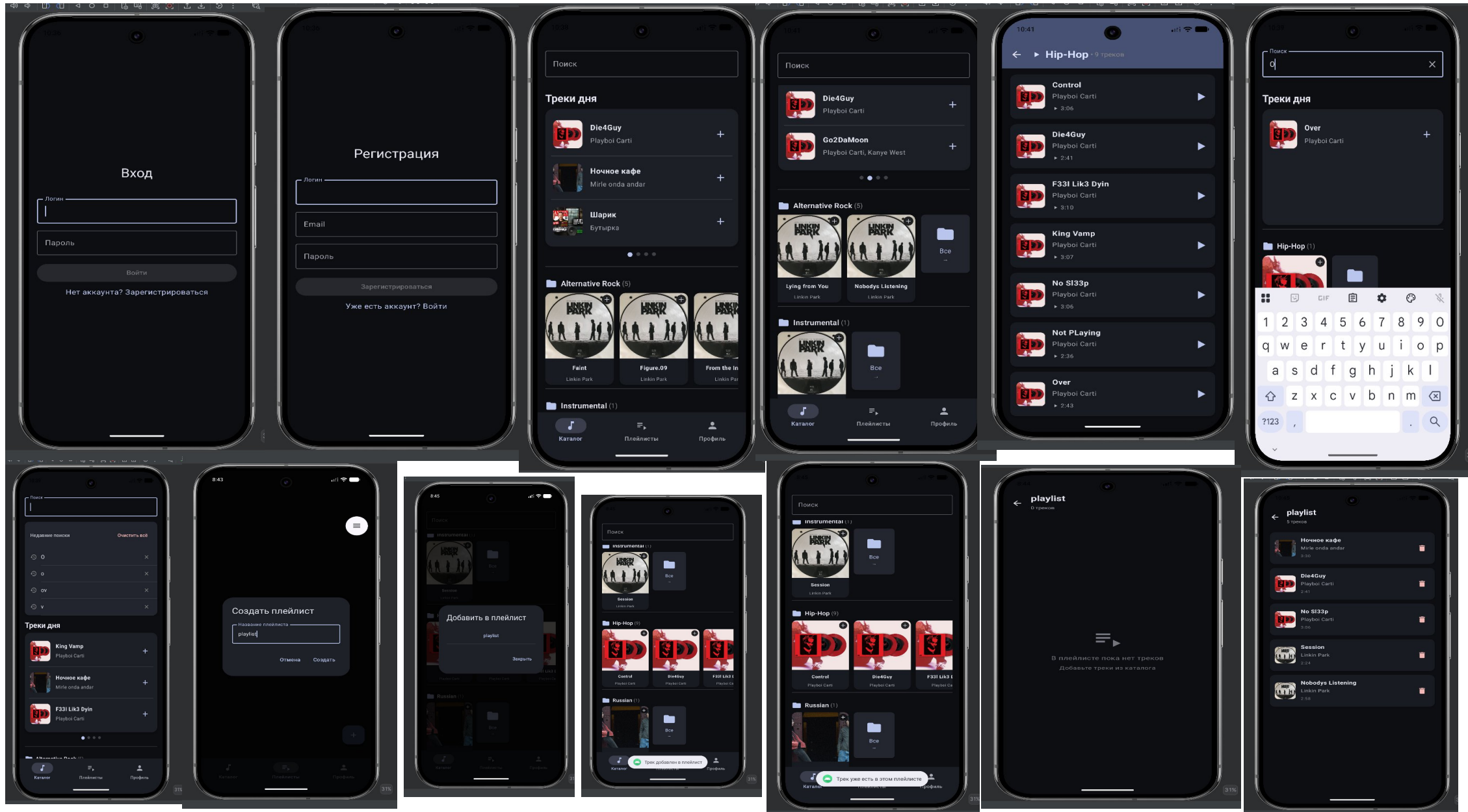
Логическая модель БД



Тестирование разработанного проекта



Демонстрация работы приложения



Заключение

- В ходе выполнения курсовой работы было разработано клиент-серверное мобильное приложение «MuseFlow» для прослушивания музыки и управления плейлистами, позволяющее пользователям искать треки, создавать собственные плейлисты, добавлять и удалять треки, а также слушать музыку в офлайн-режиме благодаря кэшированию.
- Архитектура системы включает Android-клиент на Kotlin с использованием Jetpack Compose и серверную часть на Ktor с базой данных PostgreSQL. Приложение ориентировано на простоту использования, чистую структуру и расширяемость.
- Функциональные возможности приложения включают регистрацию, авторизацию с JWT-токенами, отображение каталога треков, поиск с сохранением истории (до 10 элементов), создание и удаление плейлистов, добавление и удаление треков из плейлистов, воспроизведение музыки через ExoPlayer с фоновым режимом и медиа-уведомлением, а также переключение светлой и тёмной темы.
- Серверная часть использует Exposed ORM для взаимодействия с базой данных PostgreSQL, обеспечивает JWT-аутентификацию с хешированием паролей BCrypt и предоставляет открытое REST API. Тестирование показало стабильную работу приложения и корректную реализацию всех пользовательских сценариев.